

Justificación de los patrones

Cada patron propuesto cubre una necesidad concreta del sistema: reservas reales, pagos intercambiables, precios dinamicos y persistencia/historial.

Creacional - Factory Method

Aspecto	Detalle
Feature nueva o mejora principal	Creacion formal de reservaciones confirmadas desde backend.
Justificacion del uso	Centraliza la creacion de habitaciones, decoradores y reservas para que HotelService no concentre tanta responsabilidad de construccion.
Problema que resolveria	La reserva hoy se genera localmente en el frontend y no existe como entidad formal del backend.
Donde se implementaria	ReservaFactory, HabitacionFactory, ExtraDecoratorFactory, ReservaService y endpoint POST /hotel/reservas.
Relacion con Decorator y Bridge	Decorator calcula habitacion + extras; Factory Method crea una reserva consistente con ese resultado; Bridge usa la reserva para generar el recibo.
Nivel de complejidad	Bajo a medio

Estructural - Adapter

Aspecto	Detalle
Feature nueva o mejora principal	Multiples metodos o proveedores de pago: demo, tarjeta, PayPal o transferencia.
Justificacion del uso	Permite que el sistema use una interfaz propia de pagos mientras cada adaptador traduce a la API del proveedor correspondiente.
Problema que resolveria	Evita acoplar la logica de reserva a Stripe, PayPal u otro proveedor externo.
Donde se implementaria	PaymentProvider, PaymentService, MockPaymentAdapter, StripePaymentAdapter, PaypalPaymentAdapter y POST /hotel/reservas/:id/pagar.
Relacion con Decorator y Bridge	Decorator y Strategy determinan el total; Adapter procesa el pago; Bridge entrega el recibo despues del resultado.
Nivel de complejidad	Medio

Comportamiento - Strategy

Aspecto	Detalle
Feature nueva o mejora principal	Precios dinamicos por temporada, cupones, promociones o tarifa corporativa.
Justificacion del uso	Encapsula reglas de precio intercambiables y evita llenar HotelService.cotizar de condicionales.
Problema que resolveria	El calculo actual es fijo: costo base + extras, por lo que no escala bien ante reglas comerciales nuevas.
Donde se implementaria	PricingStrategy, PrecioBaseStrategy, TemporadaAltaStrategy, CuponDescuentoStrategy y TarifaCorporativaStrategy.
Relacion con Decorator y Bridge	Decorator genera el subtotal; Strategy calcula el total final; Bridge puede mostrar la regla aplicada en el recibo.
Nivel de complejidad	Medio

Arquitectura - Repository

Aspecto	Detalle
Feature nueva o mejora principal	Persistencia e historial de reservaciones como consecuencia de separar acceso a datos.
Justificacion del uso	Separa la logica de negocio de la fuente de datos y permite cambiar de memoria a base de datos sin tocar controladores.
Problema que resolveria	Los catalogos estan hardcodedos y no se puede consultar una reserva despues de salir de la pagina.
Donde se implementaria	ReservaRepository, HabitacionRepository, ExtraRepository e implementaciones en memoria o base de datos.
Relacion con Decorator y Bridge	Factory crea la reserva; Repository la guarda; Bridge puede regenerar recibos desde reservas persistidas.
Nivel de complejidad	Medio

Compatibilidad

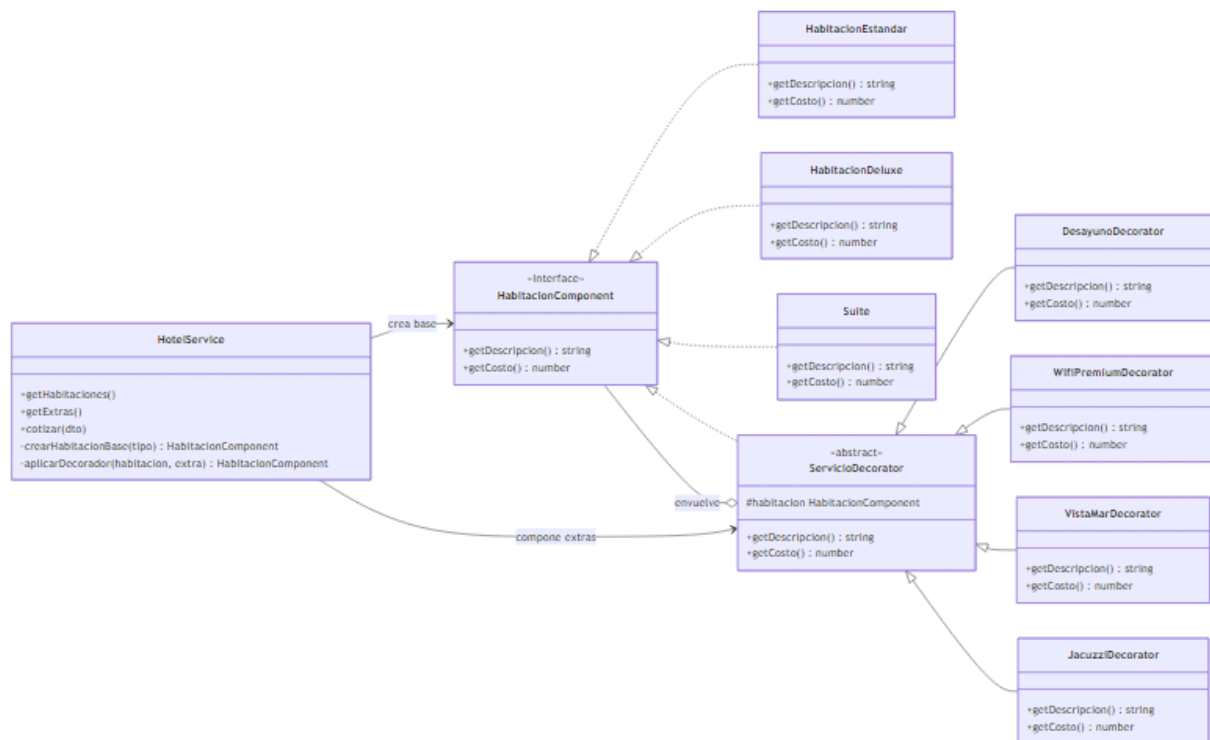
Los patrones son compatibles porque cada uno opera en una etapa distinta del flujo. La comunicacion entre ellos se da mediante resultados claros: cotización, reserva, pago y recibo.

- Decorator conserva su rol: componer habitacion base con extras y obtener descripcion/subtotal.

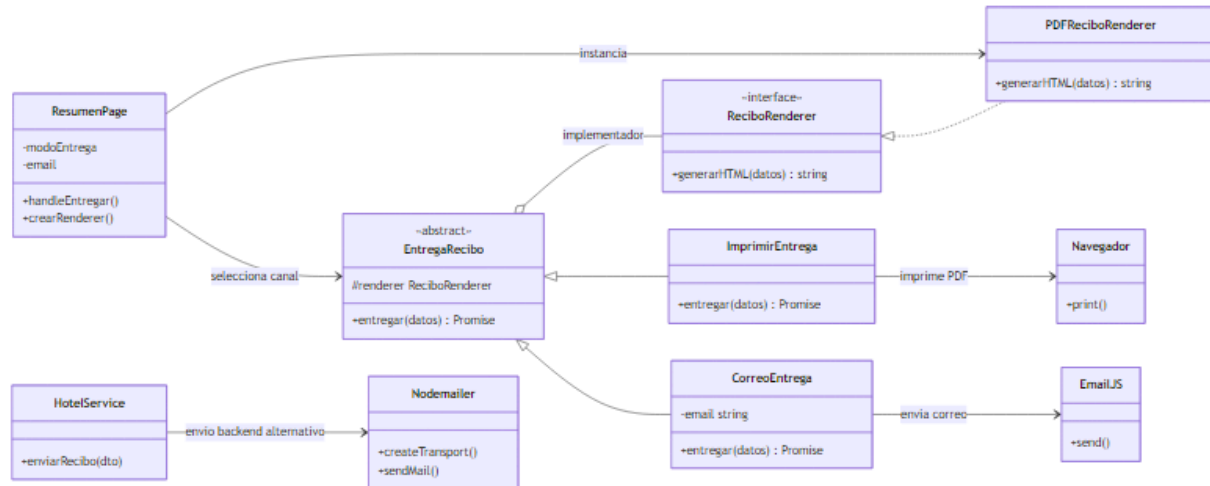
- Strategy se ejecuta despues de Decorator para ajustar el precio con reglas dinamicas.
- Factory Method crea reservas usando la cotizacion resultante; no calcula precios ni envia recibos.
- Adapter queda en la frontera externa de pagos y no contamina la logica de cotizacion o reserva.
- Repository organiza persistencia e historial sin modificar la logica de Decorator, Strategy o Bridge.
- Bridge sigue entregando recibos; puede usar datos temporales de cotizacion o reservas guardadas por Repository.

Diagramas UML

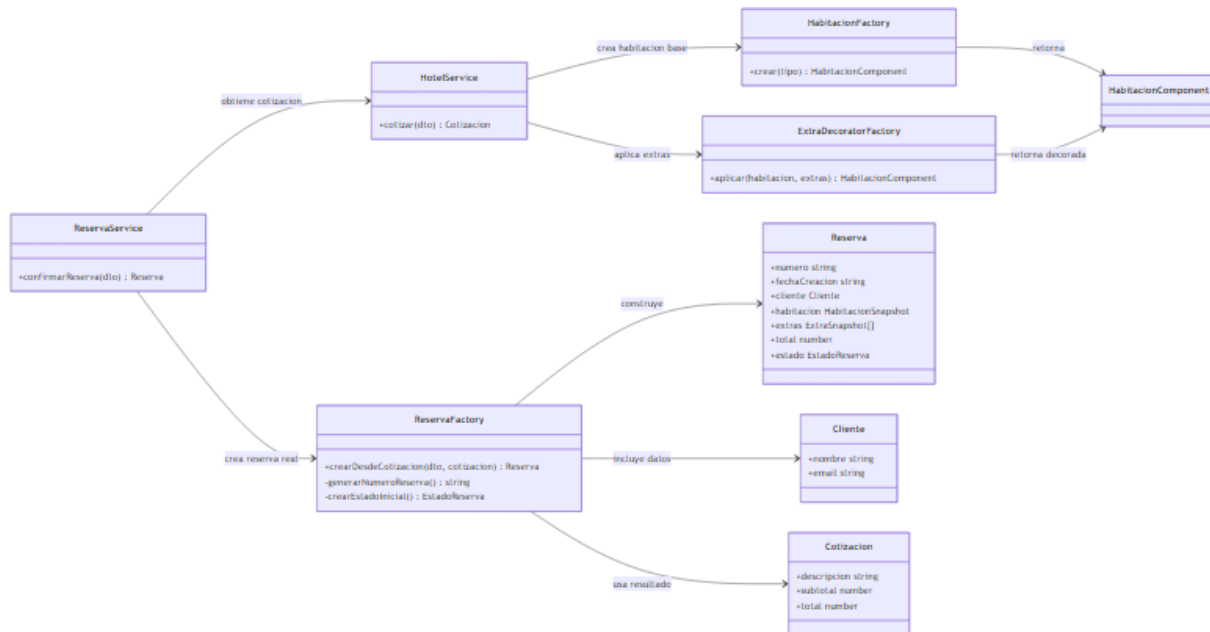
UML del patron existente: Decorator



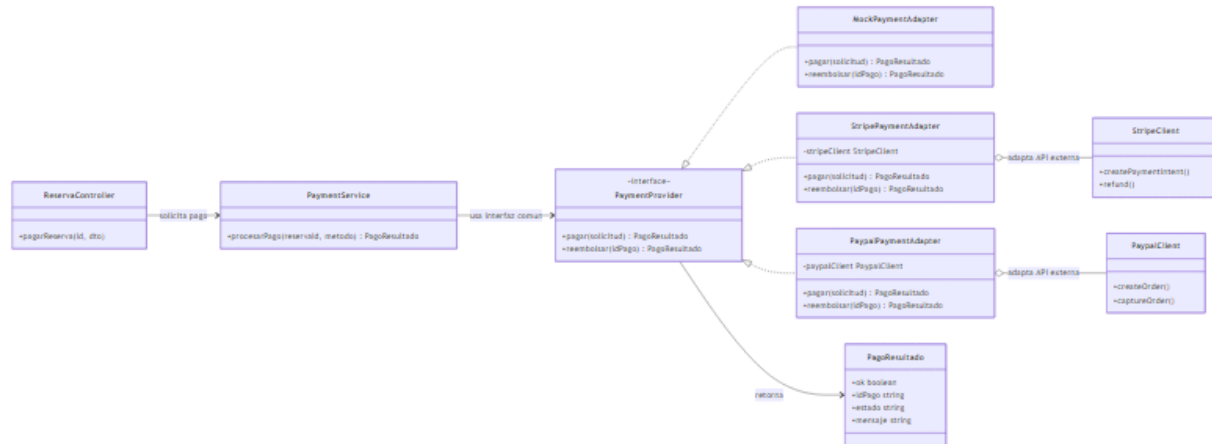
UML del patron existente: Bridge



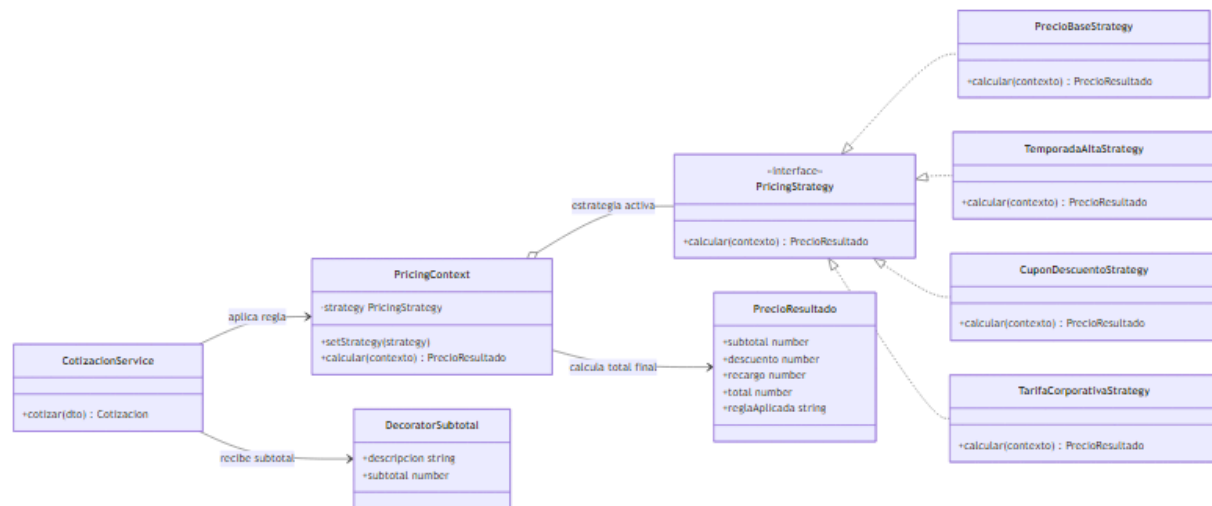
UML del patron creacional: Factory Method



UML del patron estructural: Adapter



UML del patron de comportamiento: Strategy



UML del patron de arquitectura: Repository

